

Tennis Match Analysis from a Single Broadcast Video

Court Geometry, Player Tracking, and Shot Classification

Marco Massa Alfonso Antognozzi Davide Cabitza

Signal, Image and Video Processing — Project Report, A.Y. 2025–2026

Abstract

Broadcast tennis is full of tactical information: where players stand, how far they run, which shots they choose. That information sits in the pixel domain and is normally recovered with calibrated multi-camera rigs or wearable sensors. We present an end-to-end system that converts a single, uncalibrated fixed-camera clip into structured match data, namely the court geometry, the metric trajectories of both players, the ball track, and a shot-by-shot breakdown (forehand/backhand; flat/slice/dropshot/lob; serve/smash). The design follows one principle, reconstruct the geometry once and let everything else follow from it. Eight court keypoints are detected from the white line markings with a colour mask, a probabilistic Hough transform, and intersection clustering, and a single Random Sample Consensus (RANSAC) homography maps image pixels to court metres. Every later quantity, from court positions to speeds in km/h to shot placement, is derived through that homography. Players are segmented with a running-average background model and kept identity-stable across net crossings by a minimum-cost association. The ball, being small and fast, is the only stage that uses a learned detector (YOLOv8). Shots are detected from the ball’s vertical kinematics near a player and classified with perspective-invariant, scale-free trajectory-shape features. The seven stages run as one headless pipeline with per-step error isolation, and they are validated by reproducible self-tests (shot-type classification at $21/23 = 91.3\%$ on a hand-labelled rally) and an Intersection-over-Union (IoU) evaluation against manual ground truth. The system runs on commodity footage, with no wearables and no training for any geometric stage.

1 Introduction

Extracting tactical statistics from tennis video usually depends on instrumented courts, multiple synchronised cameras, or player-worn sensors. This project takes the opposite route. A single, uncalibrated broadcast clip turns out to be enough to recover a useful match summary using mostly classical signal-, image-, and video-processing (SIV) techniques.

One idea drives the design: reconstruct the geometry once, and everything follows. A tennis court has standardised dimensions defined by the International Tennis Federation (ITF), so the four court corners and four service-box corners over-determine a planar homography from image pixels to real-world metres. Once that mapping exists, player pixel positions become court positions, pixel displacements become physical speeds, and ball-racket contacts become shots placed on a true top-down map.

The work makes four contributions. The first is a training-free court-detection front end that recovers eight labelled keypoints and, in particular, reconstructs the thin, washed-out far baseline typical of broadcast footage by a projective-consistency test. The second is a two-player background-subtraction tracker whose minimum-cost association preserves player identities through net crossings. The third is a shot-analysis stage that detects contacts from ball kinematics and classifies stroke, shot type, and overhead category from perspective-invariant features, so the foreshortened far half of the court needs no per-side calibration. The fourth is the engineering itself: a headless, single-command pipeline with per-stage error isolation, a stable inter-stage data contract, reproducible self-tests, and a quantitative evaluation against manual annotations.

Table 1 summarises the recognition vocabulary. Only the ball detector is learned; court detection, player tracking, motion estimation, and every shot decision are deterministic, inter-

Table 1: Recognition layers and decision cues.

Layer	Classes	Decision cue
Stroke	fore/back/unknown	body side struck (perspective)
Shot type	flat/slice/drop/lob	outgoing trajectory shape
Overhead	serve/smash	incoming vertical direction

Table 2: The seven pipeline stages (classical unless noted).

#	Stage / method	Output
1	Court: mask, Hough, RANSAC	<code>*_court.csv</code>
2	Players: bg-sub, min-cost assoc.	<code>players_*.csv</code>
3	Player analysis: homography stats	heatmaps, zones
4	Motion: optical flow, block match	flow fields, PSNR
5	Ball: YOLOv8, interpolation	<code>ball_*.csv</code>
6	Shots: kinematics, classifiers	<code>shots.csv</code>
7	Evaluation: IoU vs. ground truth	error metrics

pretable, and tunable. The test sequence used throughout (`Input_video2`) is a broadcast clip of an Alcaraz–Sinner rally (Miami 2023): 1008 frames at 30 frames per second (fps), about 33.6 s.

2 Background and Related Work

The pipeline rests on standard SIV building blocks. Colour-space thresholding in Hue–Saturation–Value (HSV) space isolates the white court lines, exploiting the separation of chroma from luminance that HSV gives over RGB. Straight court lines are recovered with the probabilistic Hough transform, which accumulates collinear edge votes and returns finite line segments. The planar homography that maps the court to metres is the projective transform between two views of a plane, estimated with RANSAC so that a single mislabelled keypoint is absorbed by the largest consensus set.

Foreground extraction relies on running-average background subtraction, a recursive temporal filter $B_t = (1 - \alpha)B_{t-1} + \alpha I_t$ that adapts a per-pixel background model. Morphological opening and closing then clean the binary mask, and connected-component analysis extracts the blobs. Associating those blobs across frames is a small linear-assignment problem, solved here by exhaustive enumeration because there are only two targets.

Motion estimation is the core video-processing material of the course, and the pipeline uses two classical families. Optical flow is covered by dense Farnebäck flow and by pyramidal Lucas–Kanade tracking of Shi–Tomasi corners. Block matching is covered by full and three-step search, scored by the Peak Signal-to-Noise Ratio (PSNR) of the motion-compensated prediction. The ball is the exception. Being small, fast, and motion-blurred, it is detected by a learned YOLOv8 single-stage detector, since a purely classical detector is unreliable at that scale. Its track is differentiated through a Savitzky–Golay filter, which fits a local low-order polynomial and so preserves the velocity and acceleration peaks that mark a contact, where a moving average would smear them.

3 System Overview

The pipeline runs as seven sequential stages. Each stage consumes the artefacts of the previous ones and writes its own Comma-Separated Values (CSV) file and figures, so any stage can be run, inspected, or re-run on its own. The pixel-to-metre homography is not itself a stage but a shared utility (`utils/court_converter.py`) that every metric stage calls, from player analysis to motion estimation to shot analysis to the live viewer. The raw video feeds the tracking, motion, ball, and shot stages directly. Table 2 lists the stages, their methods, and their outputs.

4 Methodology

4.1 Court detection and geometry

The court lines are detected first, straight from the white markings and with no training (`court_tracking.py`). The first frame is converted to HSV and thresholded with bounds $(0, 0, 180)$ – $(180, 50, 255)$ to isolate the painted lines. A region-of-interest mask then removes the top fraction of the frame (default 0.15), where banners and fences would otherwise contribute spurious white. The mask is cleaned with a 3×3 morphological close (two iterations), and the result is passed through the Canny detector and the probabilistic Hough transform (`cv2.HoughLinesP`, threshold 150, minimum length 80 px, maximum gap 30 px).

Candidate corners are the pairwise intersections of the detected segments. For two segments with endpoints (x_1, y_1, x_2, y_2) and (x_3, y_3, x_4, y_4) , the intersection is

$$\mathbf{p} = \mathbf{p}_1 + t(\mathbf{p}_2 - \mathbf{p}_1), \quad (1)$$

$$t = \frac{(x_1 - x_3)(y_3 - y_4) - (y_1 - y_3)(x_3 - x_4)}{(x_1 - x_2)(y_3 - y_4) - (y_1 - y_2)(x_3 - x_4)}, \quad (2)$$

computed only for pairs whose acute angle exceeds 25° . Intersections within 50 px are merged into one centroid by a deterministic, order-independent clustering pass.

Two domain-specific filters then isolate the singles court. The ITF-proportion filter discards the doubles-alley corners. Within each horizontal row of dots, it predicts the singles-corner positions from the doubles span using the known alley ratio $\rho = 1.37/10.97 \approx 0.125$ and keeps the dot nearest each prediction. The far-line recovery handles a failure peculiar to broadcast footage. The far baseline is thin and washed out, so rather than trust the Hough corners there, the method fits each sideline by least squares ($x = ay + b$), scans a one-dimensional white-coverage profile of every row between the sidelines, and chooses the (baseline, service-line) row pair that is projectively consistent with the ITF court model, that is, the pair whose induced homography lands the remaining service lines on detected bands within a 12 px residual. The elevated net band and banner edges fail that test and are rejected automatically.

The eight labelled keypoints (court corners `TL, TR, BL, BR`; service-box corners `STL, STR, SBL, SBR`) are written to CSV. A single RANSAC homography $\mathbf{H}$ (`cv2.findHomography`, reprojection threshold 3 px) maps the image keypoints onto the ITF singles court ($8.23 \text{ m} \times 23.77 \text{ m}$, with service lines 18 ft from each baseline):

$$\begin{bmatrix} x_m \\ y_m \\ w \end{bmatrix} = \mathbf{H} \begin{bmatrix} x_{px} \\ y_{px} \\ 1 \end{bmatrix}, \quad (x_m, y_m) = \left(\frac{x_m}{w}, \frac{y_m}{w} \right). \quad (3)$$

RANSAC keeps the fit stable when one keypoint is mislabelled, and a point near the horizon (a small homogeneous divisor w) is guarded and returned as Not-a-Number (NaN) rather than $\pm\infty$. This homography is the backbone of the project: every metric quantity downstream is produced through it.

4.2 Player tracking

Players are segmented by background subtraction with identity-stable association (`playerTracking.py`). A running-average background is maintained with learning rate $\alpha = 0.15$. That model takes time to converge, so a static reference background is built once, as the temporal median of the first second of frames. During the warm-up window the tracker subtracts against this median while the running-average model converges alongside it and takes over once warm-up ends, which means a poor warm-up seed can never corrupt the reliable model.

Each frame’s foreground mask is the thresholded absolute difference against the background, $|I_t - B_t| > \tau$ with $\tau = 15$ (kept independent of frame rate), followed by Gaussian smoothing, a

morphological open with a small kernel, and a close with a large one. Connected components below 300 px² are gated out as noise.

Identity preservation is the hard part. When two identities are already tracked, the two current detections are assigned by the assignment of minimum total displacement over the 2×2 options: among all ordered candidate pairs (c_i, c_j) , the method minimises

$$\text{cost} = \|c_i - \hat{p}_1\| + \|c_j - \hat{p}_2\| \quad (4)$$

subject to a per-player maximum-displacement gate. This is what stops P1 and P2 from swapping labels when they cross near the net, where a greedy nearest-neighbour matcher flips them. Because the far player is small, candidates are matched by displacement over all blobs rather than the two largest by area, and identity loss and re-acquisition are logged.

4.3 Player analysis

Each player’s feet point $(c_x, y + h)$, the bottom-centre of the bounding box, is projected to metres through the homography (`player_analysis.py`). The feet point is used instead of the centroid because the body centroid sits roughly a metre above the ground plane and would project too far from the net. Per-frame speed between consecutive detections is

$$v = \frac{\|\mathbf{x}_m(t+1) - \mathbf{x}_m(t)\|}{\Delta t} \times 3.6 \text{ [km/h]}, \quad (5)$$

and any displacement implying more than 45 km/h is discarded as a tracking glitch. From these the stage produces per-player speed statistics (mean, median, 95th percentile, maximum) and total distance, court-zone occupancy over a 6×4 grid that covers the walkable run-off, Gaussian-smoothed position heatmaps per player and combined, and an animated top-down minimap.

4.4 Ball tracking

The ball is tiny, fast, and often motion-blurred, the one place where a learned detector earns its keep (`BallTracking.py`). A YOLOv8 detector runs per frame at confidence 0.65. Each candidate, taken in descending confidence order, must pass a frame-difference motion gate: at least 5% of the pixels inside the candidate box must change by more than an intensity of 15 between consecutive grayscale frames. The first candidate that passes is accepted, which rejects static false positives such as line markings and logos. Short detection gaps are filled by linear interpolation, and an `interpolated` flag is written to the CSV so later stages can tell real detections from synthetic fills. That flag matters: a long straight-line fill meeting real motion otherwise mimics a racket contact.

4.5 Shot detection and classification

Contact detection. A shot is a sudden change in the ball’s vertical motion right next to a player. On the Savitzky–Golay-smoothed track (window 9, order 2), velocities and accelerations are computed per contiguous valid run, so a one-frame dropout does not contaminate its neighbours. A candidate at frame i needs a persistent sign reversal of the vertical velocity v_y (with $\min(|\bar{v}_y^{\text{bef}}|, |\bar{v}_y^{\text{aft}}|) \geq 0.5 \text{ px/frame}$), or a local peak of the acceleration magnitude $|\mathbf{a}| \geq 1.5 \text{ px/frame}^2$ confirmed by an independent post-contact reversal check, together with the ball lying inside the striking player’s expanded bounding box. Bounces also reverse v_y , but they happen away from the players, so the proximity gate removes them. Candidates from one physical contact are merged by wide-gap clustering, keeping the member with the strongest acceleration.

Stroke (forehand/backhand). At the contact frame, the ball’s horizontal offset from the player’s body axis decides the stroke, with a dead-band (12% of box width) labelled *unknown*. The decision is perspective-aware. The near player is seen from behind, so their right is image-right, whereas the far player faces the camera, so their right is image-left, and a left-hander inverts the mapping.

Shot type (flat/slice/dropshot/lob). The angled camera makes a near-player pixel speed roughly three times a far-player one for the same physical shot, so raw pixel speed is never compared across sides. Perspective is neutralised in three ways. The first is a scale-free *pace*, which divides the mean per-step pixel speed by the local median ball-box height $\tilde{h}$:

$$pace = 100 \cdot \overline{\|\Delta \mathbf{x}_{px}\|} / \tilde{h}. \quad (6)$$

The second is a set of approximate metric speeds from the homography, used only for the far flat/slice split. And dimensionless shape features capture the rest: **diefrac** (second-half over first-half outgoing path length), **reach** (path length over $\tilde{h}$), and **bowback**, the fraction of the vertical range the ball travels back after an interior extreme, near 0 for a flat drive and high for a lofted arc. A short, ordered rule set then assigns the type. A far ball whose path collapses with low late speed is a dropshot. A near ball with tiny **diefrac**, slow pace, and short reach is a lob. A near ball that bows back and loses pace is a slice. A far ball is flat when its peak metric speed is high and a slice otherwise; everything else is flat. The thresholds were fit against a 23-shot hand-labelled rally, sit on wide plateaus, and are exposed as command-line flags for per-camera retuning.

Overhead (serve/smash). Serve and smash live in a separate, additive overhead column, so they never disturb the flat/slice/dropshot/lob labels, and the classifier is deliberately cautious, returning empty whenever a signal is missing so that a groundstroke is never relabelled. The common geometric gate requires the ball clearly above the box top and horizontally aligned with the player. The two are then told apart by the vertical direction of the incoming ball, a scale-free cue that survives the ground homography distorting an airborne ball. A serve is the first shot, with the player stationary behind their own baseline and the incoming toss rising. A smash is struck off a ball arriving fast and descending, driven downward with a low apex. The overhead label takes precedence in the combined category, so a detected serve is drawn as a serve whatever its (inapplicable) groundstroke side.

4.6 Motion estimation

Two classical techniques cross-check player speed and measure inter-frame motion (**motionEstimation/**). Optical flow uses dense Farnebäck flow (pyramid scale 0.5, three levels, window 15), whose mean flow inside each player box, converted to km/h through the homography, is compared with the position-based speed, alongside a sparse Lucas–Kanade demonstration that tracks Shi–Tomasi corners. Block matching is implemented from scratch over a sum-of-absolute-differences (SAD) criterion, with both exhaustive full search and the logarithmic three-step search. For each frame pair it estimates a motion-vector field on 16×16 blocks, builds the motion-compensated prediction, and reports its PSNR against the no-motion baseline,

$$\text{PSNR} = 10 \log_{10} \frac{255^2}{\text{MSE}}. \quad (7)$$

4.7 Evaluation

Correctness is measured against manual ground truth (**evaluation/**). A small annotation tool produces ground-truth player boxes and court keypoints. The evaluator matches predicted and ground-truth boxes per frame by best total IoU,

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}, \quad (8)$$

and reports mean and median IoU, precision and recall at an IoU threshold of 0.5, centre error in pixels, feet error in metres through the homography, assignment-level ID switches, and per-keypoint court error.

Table 3: Self-test results (`shot_analysis` CLI flags).

Self-test	Result
<code>--self-test</code> (synthetic strokes, incl. lefty)	4/4
<code>--type-self-test</code> (23-shot GT, shot type)	21/23 = 0.913
<code>--overhead-self-test</code> (serve/smash)	4/4

5 Implementation

The system is written in Python 3 and organised as importable packages: `tracking/` for the court, player, and ball trackers, `utils/` for the `CourtConverter` homography utility, player analysis, and shot analysis, `motionEstimation/` for optical flow and block matching, and `evaluation/` for annotation and metrics. It uses OpenCV for image and video operations, NumPy for array computation, pandas for CSV handling, SciPy for the Savitzky–Golay filter and Gaussian smoothing, Matplotlib for figures (pinned to the headless `Agg` backend), and Ultralytics YOLOv8 for ball detection.

`pipeline.py` is the single entry point. It reads the frame rate from the video, derives every artefact path from the video stem, and runs all seven stages from one command. It is headless by default, with `--display` enabling OpenCV windows. Two choices keep it from falling over. Every step sits inside its own error guard, so one failure is reported and skipped rather than aborting the run, and any step can be skipped with a `--skip-*` flag to reuse an existing CSV. A closing summary lists which steps succeeded, were skipped, or failed.

A companion viewer, `live_view.py`, replays the result in a single window: the original clip with player and ball overlays, a synchronised top-down minimap, a live shot-statistics panel, and an end-of-clip summary. It only displays; it reads the CSVs the pipeline produced and never re-runs a tracker. One habit recurs through the codebase, an append-only inter-stage CSV contract. The ball `interpolated` flag, for instance, is added as the last column, so a newer producer never breaks an older reader. Together with per-stage isolation, that is what let shot analysis and the unified pipeline arrive late without destabilising the detectors already in place.

6 Experiments and Results

The pipeline was run end-to-end on the `Input_video2` test clip (1008 frames, 30 fps), and it produces a complete, coherent match summary.

For the court, eight keypoints are detected and a homography is fitted with low (about 1 px) line-fit residuals on the detected court lines. Both players are tracked across the full clip with stable identities through the net crossings, and the rare identity-loss events are logged. Shot detection finds 25 contacts, 13 by the near player (P1) and 12 by the far player (P2). The opening serve is among them, correctly labelled `serve` in the overhead column (frame 11, $t = 0.37$ s) and drawn as a serve on the hitmap. On the orthogonal groundstroke axis the contacts split into 18 backhands and 6 forehands, with the serve marked `unknown` there because it is not a groundstroke. By type they are 17 flat, 5 slice, 2 dropshot, and 1 lob, with no smash in this clip. Against the hand-labelled rally the shot-type classifier scores $21/23 = 91.3\%$ (Table 3), above the 90% acceptance gate and stable across a wide threshold range.

The classifiers ship with reproducible self-tests that double as regression guards (Table 3). The synthetic stroke test and the synthetic overhead test both pass 4/4, and the overhead test confirms zero false positives, since a synthetic groundstroke and a synthetic lob both produce no overhead label.

The qualitative outputs match the expected match structure. The combined heatmap (Fig. 1) shows the near player covering the baseline corners while the far player is compressed by perspective, and the zone occupancy (Fig. 2) puts numbers on it: the near player spends 53.4% of frames

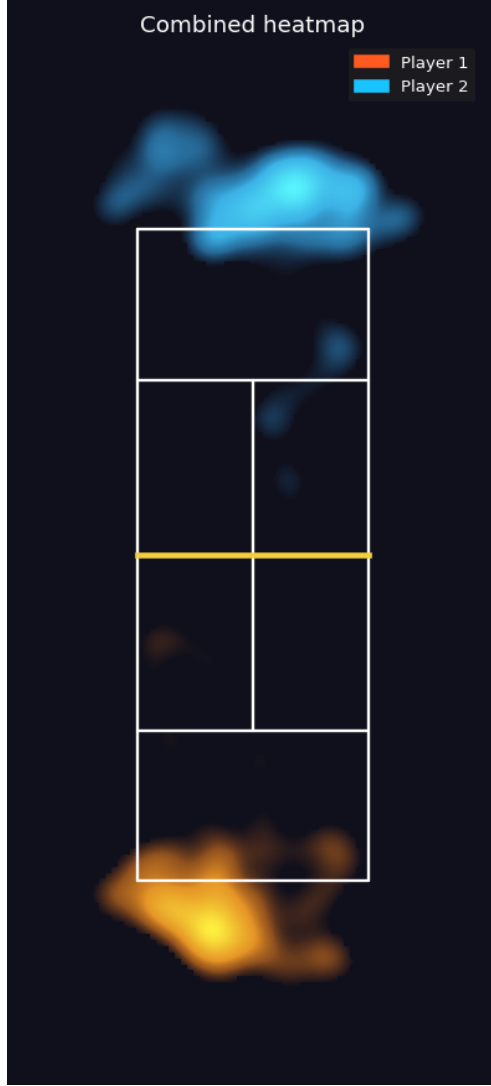


Figure 1: Combined position heatmap (P1 warm, P2 cool). The near player covers the baseline corners; the far player is perspective-compressed. Generated by `player_analysis.py`.

behind the near baseline on the left, and the far player 44.4% and 23.8% behind the far baseline. The sparse flow field drawn on the frame (Fig. 3) makes the same point at the level of individual vectors: arrows pile up on the two players and leave the court empty. The shot hitmap (Fig. 4) places every shot at the player’s court position at contact, coloured by category, with the opening serve sitting correctly behind the near baseline. The dense optical flow (Fig. 5) separates the moving players and ball from the static court, and the block-matching stage reports a positive PSNR gain of the motion-compensated prediction over the no-motion baseline.

7 Discussion

The system has four properties worth drawing out. It is mostly training-free: only the ball uses a learned model, while the court, players, motion, and shot reasoning stay deterministic and interpretable. It is metric by construction, because the single homography makes every downstream quantity physical, whether metres, km/h, or court zones. Its shot reasoning is perspective-invariant, since scale-free features and direction-based overhead cues avoid comparing pixel speeds across the foreshortened far half, the main source of error in single-camera tennis analysis. And it is built for safety, through per-step error isolation, a stable inter-stage CSV contract, and self-tests that act as regression guards. The overhead classifier is the clearest example of that last point: a separate, conservative column guarded by a zero-false-positive synthetic test, added precisely so

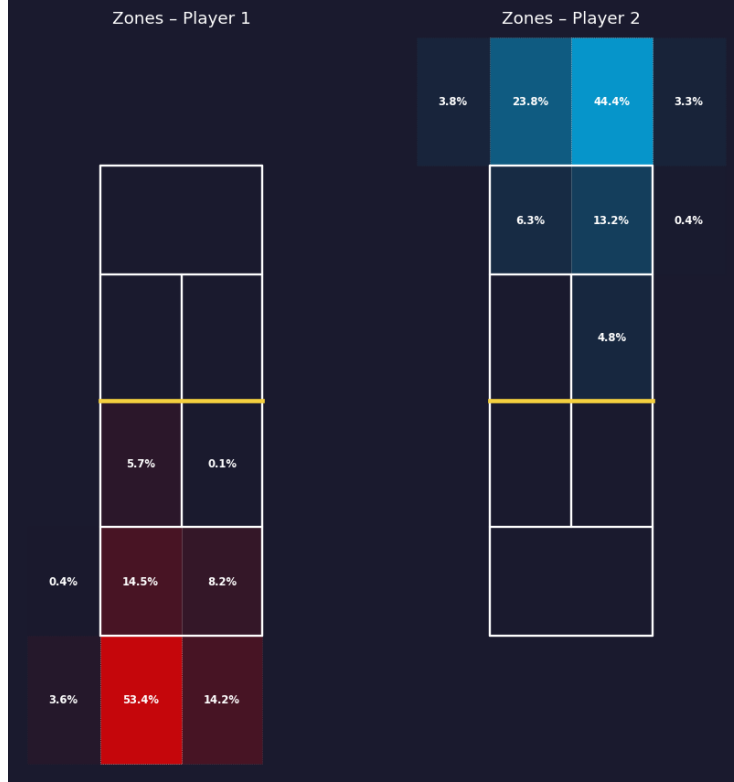


Figure 2: Per-player court-zone occupancy over the 6×4 grid, overlaid on the court (`player_analysis.py`).

that rarer shot categories could not damage the groundstroke labels already working.

The limitations are concrete. The homography assumes a single fixed camera, so a camera cut would force the court to be detected again. Shot-analysis quality is bounded by the ball track, because long YOLO dropouts force interpolation that can blur a contact frame, and the two shot-type misclassifications fall on exactly the slow-ball cases the ground truth itself marks “dropshot/slice”. The court homography is a ground plane, so the projected position of an airborne ball is only approximate, which is why shot reasoning leans on scale-free and direction features rather than raw metric ball coordinates. A serve in the opening frames depends on the player tracker covering the clip start, so a truncated player CSV can miss it. Finally, the shot-type thresholds were tuned on one rally and may need per-camera retuning, though all are exposed as flags and the scale-free cuts transfer better than the metric ones.

8 Conclusion and Future Work

A single, uncalibrated tennis broadcast clip can be turned into structured match data with a mostly classical computer-vision pipeline. By reconstructing the court geometry once and propagating it through every stage, the system delivers player tracking, movement statistics, motion estimation, and a shot-by-shot breakdown that includes serve and smash, all from commodity footage and with reproducible validation: $21/23 = 91.3\%$ shot-type accuracy and an IoU-based tracking evaluation.

The limitations point to the next steps. Camera-cut handling would re-detect the court across shots and open the system to full-match footage. A learned shot classifier trained on a larger labelled set could replace the hand-tuned thresholds while keeping the interpretable features. The detected serves already give a foothold for rally and point segmentation. Temporal models for tracking and shot detection would absorb ball dropouts and occlusions better than the current per-frame logic. Broader validation across matches, players, and broadcast styles would then show how far the scale-free design carries.

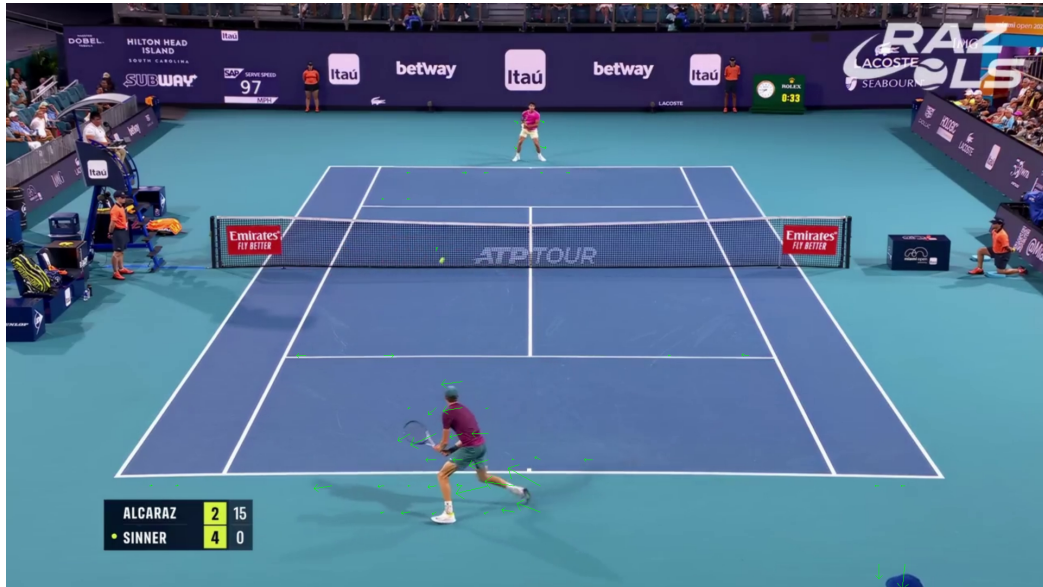


Figure 3: Sparse optical-flow field drawn on the source frame: each arrow is the mean Farnebäck flow on a grid cell, so the vectors concentrate on the two moving players while the static court stays clear (optical_flow.py).

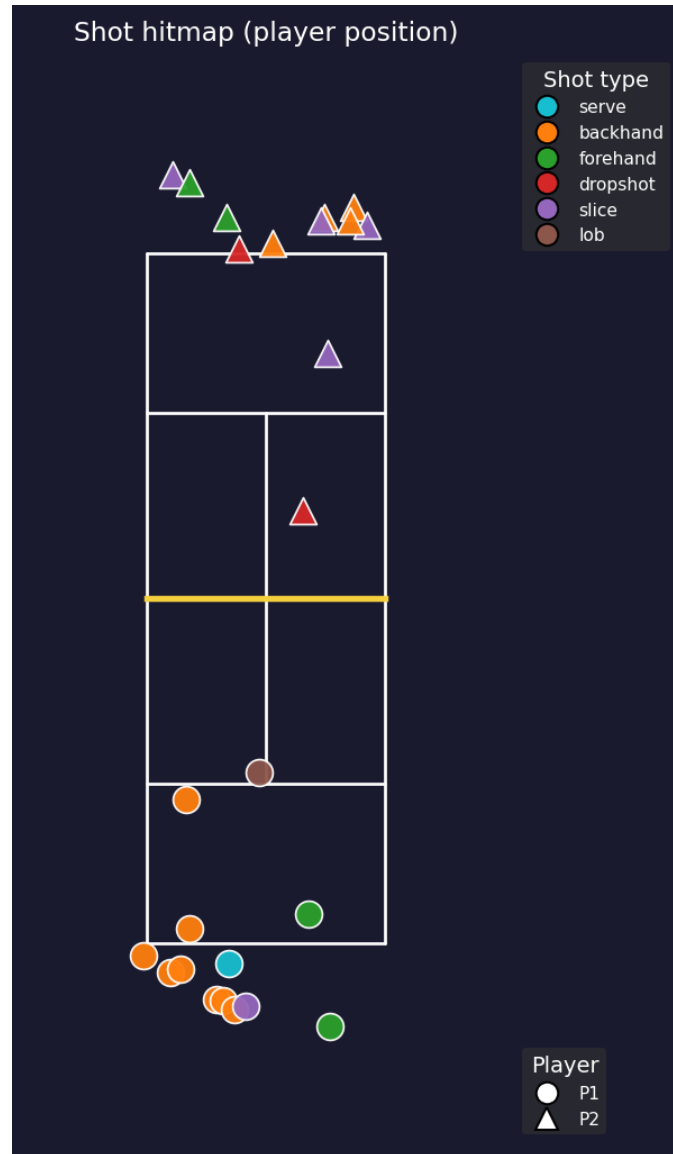


Figure 4: Shot hitmap: each shot at the player’s court position at contact, coloured by category. P1 = circles (near), P2 = triangles (far); the opening serve (cyan) sits behind the near baseline (`shot_analysis.py`).

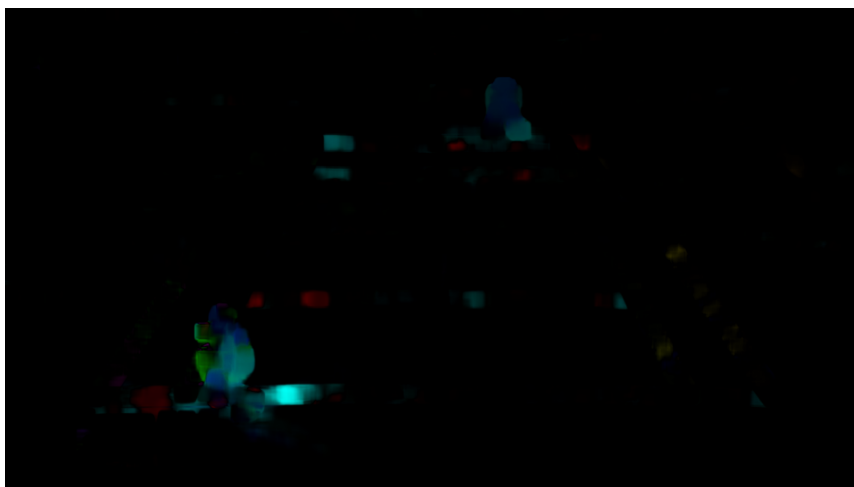


Figure 5: Dense Farnebäck optical flow: hue encodes direction, brightness encodes magnitude (`optical_flow.py`). Moving players and ball stand out against the static court.